15. 3Sum Medium

Acceptance: 38.4% | LeetCode

Given an integer array nums, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

Example 1:

```
**Input:** nums = [-1,0,1,2,-1,-4]
**Output:** [[-1,-1,2],[-1,0,1]]
**Explanation:**
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of
the triplets does not matter.
```

Example 2:

```
**Input:** nums = [0,1,1]
**Output:** []
**Explanation:** The only possible triplet does not
sum up to 0.
```

Example 3:

```
**Input:** nums = [0,0,0]
**Output:** [[0,0,0]]
**Explanation:** The only possible triplet sums up to
0.
```

Constraints:

- `3 <= nums.length <= 3000`
- `-105 <= nums[i] <= 105`

Claude Opus Solution

3Sum — Complete Solution Guide

1. Intuition

The naive approach checks every triplet in **O(n³)** time. The key insight is:

> **Sort the array first.** Once sorted, we can fix one element and use the classic **two-pointer technique** to find the other two elements in linear time.

This reduces the inner search from O(n²) (nested loops) to O(n), giving us **O(n²)** overall.

Why does sorting help with the two pointers? - If the sum of three numbers is **too small**, move the **left pointer right** (to a larger number). - If the sum is **too large**, move the **right pointer left** (to a smaller number).

This monotonic behavior only works because the array is sorted.

Handling duplicates is the trickiest part. Sorting groups equal numbers together, so we can easily skip over them to avoid duplicate triplets.

---

2. Approach (Pseudocode)

```
1. Sort nums ascending.
2. For each index i from 0 to n-1:
     a. If nums[i] > 0: break  (no triplet can sum to 0 with all positives)
     b. If i > 0 and nums[i] == nums[i-1]: skip  (avoid duplicate first element)
     c. Set two pointers: left = i+1, right = n-1
     d. While left < right:
          - total = nums[i] + nums[left] + nums[right]
          - If total == 0:
                record triplet
                move left right, right left
                skip duplicates on both sides
          - If total < 0: move left++  (need bigger sum)
          - If total > 0: move right--  (need smaller sum)
3. Return all recorded triplets.
```

---

3. Python Solution

```python
from typing import List
```

```python
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        # Step 1: Sort so we can use two pointers and skip duplicates easily.
        # Sorting costs O(n log n).
        nums.sort()
        result = []
        n = len(nums)

        # Step 2: Fix the first element of the triplet at index i.
        for i in range(n):
            # Optimization: since array is sorted, if nums[i] > 0, then
            # every element after it is also > 0, so no triplet can sum to 0.
            if nums[i] > 0:
                break

            # Skip duplicate values for the first element to avoid
            # producing the same triplet more than once.
            if i > 0 and nums[i] == nums[i - 1]:
                continue

            # Step 3: Use two pointers to find pairs summing to -nums[i].
            # We want nums[left] + nums[right] == -nums[i].
            left, right = i + 1, n - 1

            while left < right:
                total = nums[i] + nums[left] + nums[right]

                if total == 0:
                    # Found a valid triplet.
                    result.append([nums[i], nums[left], nums[right]])

                    # Move both pointers inward to look for new triplets.
                    left += 1
                    right -= 1

                    # Skip duplicates for the second element.
                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                    # Skip duplicates for the third element.
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1

                elif total < 0:
                    # Sum too small -> need a larger number -> move left right.
                    left += 1
                else:  # total > 0
                    # Sum too large -> need a smaller number -> move right left.
                    right -= 1

        return result
```

---

4. Complexity Analysis

Aspect	Complexity	Explanation
Time	**O(n²)**	Sorting is O(n log n). The outer loop runs n times; for each i, the two-pointer scan is O(n). Total: O(n log n) + O(n²) = **O(n²)**.
Space	**O(1)** or **O(n)**	Ignoring the output list, we use only constant extra space for pointers. (Sorting may use O(log n) to O(n) depending on implementation. The output itself can be O(n²) in the worst case.)

---

5. Alternative Approaches

a) Brute Force — O(n³)

Three nested loops checking all triplets, with a set to dedupe. - Simple but too slow for n = 3000.

b) HashSet for the inner Two-Sum — O(n²) time, O(n) space

Fix `i`, then iterate `j > i` using a hash set to find `-(nums[i]+nums[j])`.

```python
class Solution:
    def threeSum(self, nums):
```

```
        nums.sort()
        res = set()
        n = len(nums)
        for i in range(n):
            if i > 0 and nums[i] == nums[i-1]:
                continue
            seen = set()
            for j in range(i + 1, n):
                complement = -nums[i] - nums[j]
                if complement in seen:
                    res.add((nums[i], complement, nums[j]))
                seen.add(nums[j])
        return [list(t) for t in res]
```

- Same time complexity but uses extra space and a set for dedup. The two-pointer method is generally preferred for its **O(1)** auxiliary space.

c) No-Sort Approach

Possible using hash maps, but managing duplicates becomes far more complex and error-prone. **Sorting + two pointers remains the cleanest, optimal solution.**

Key Takeaways

- **Sort to enable two pointers** — a recurring pattern for k-Sum problems.
- **Skip duplicates** at each level to ensure unique results.
- This problem generalizes: **4Sum** uses the same idea with one more outer loop ($O(n^3)$), and **kSum** can be solved recursively reducing to 2Sum.

Codex Solution

1. Intuition

The key pattern is:

> **Sort the array, then reduce 3Sum into repeated 2Sum problems using two pointers.**

For each number `nums[i]`, we want to find two numbers after it such that:

```
nums[i] + nums[left] + nums[right] == 0
```

Equivalently:

```
nums[left] + nums[right] == -nums[i]
```

After sorting, we can use two pointers:

- If the sum is too small, move `left` rightward to increase it.
- If the sum is too large, move `right` leftward to decrease it.
- If the sum is exactly `0`, record the triplet and skip duplicates.

Sorting also makes it easy to avoid duplicate triplets.

---

2. Approach

Pseudocode

```
sort nums
result = []

for i from 0 to len(nums) - 3:
    if nums[i] > 0:
        break

    if i > 0 and nums[i] == nums[i - 1]:
        continue

    left = i + 1
```

```
    right = len(nums) - 1

    while left < right:
        total = nums[i] + nums[left] + nums[right]

        if total == 0:
            add [nums[i], nums[left], nums[right]] to result

            move left rightward
            move right leftward

            skip duplicate values for left
            skip duplicate values for right

        else if total < 0:
            move left rightward

        else:
            move right leftward

return result
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        # Sort the array so that:
        # 1. We can use the two-pointer technique.
        # 2. Duplicate triplets become easier to detect and skip.
        nums.sort()

        result = []
        n = len(nums)

        # Fix one number nums[i], then solve 2Sum on the remaining suffix.
        for i in range(n - 2):
            # Since the array is sorted, if nums[i] > 0,
            # then nums[i], nums[i+1], nums[i+2], ... are all positive.
            # Therefore, no future triplet can sum to 0.
            if nums[i] > 0:
                break

            # Skip duplicate first elements.
            # Without this, we could produce duplicate triplets.
            #
            # Example:
            # nums = [-1, -1, 0, 1]
            # If we process both -1 values as the first element,
            # we may generate the same triplet twice.
            if i > 0 and nums[i] == nums[i - 1]:
                continue

            # Now find two numbers after index i whose sum is -nums[i].
            left = i + 1
            right = n - 1

            while left < right:
                total = nums[i] + nums[left] + nums[right]

                if total == 0:
                    # Found a valid triplet.
                    result.append([nums[i], nums[left], nums[right]])

                    # Move both pointers inward to look for more pairs.
                    left += 1
                    right -= 1

                    # Skip duplicate second elements.
```

```
                    while left < right and nums[left] == nums[left - 1]:
                        left += 1

                    # Skip duplicate third elements.
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1

                elif total < 0:
                    # Sum is too small.
                    # Move left pointer rightward to increase the sum.
                    left += 1

                else:
                    # Sum is too large.
                    # Move right pointer leftward to decrease the sum.
                    right -= 1

        return result
```

---

4. Complexity Analysis

Let `n` be the length of `nums`.

Time Complexity

```
O(n^2)
```

Explanation:

- Sorting takes `O(n log n)`.
- The outer loop runs up to `n` times.
- For each fixed `i`, the two-pointer scan takes `O(n)` time.
- Therefore, the main loop costs `O(n^2)`.

Overall:

```
O(n log n + n^2) = O(n^2)
```

Space Complexity

```
O(1)
```

If we ignore the output list, the algorithm only uses a few variables.

However, Python's built-in sort may use additional internal space depending on implementation details. In LeetCode complexity discussions, this solution is commonly considered:

```
O(1) auxiliary space, excluding output
```

The output itself can contain many triplets, so output space can be up to `O(n^2)` in the worst case.

---

5. Alternative Approaches

1. Brute Force

Check every possible triplet.

```
for i:
    for j:
        for k:
            check nums[i] + nums[j] + nums[k] == 0
```

Complexity:

```
Time: O(n^3)
Space: O(1)
```

This is too slow for `n <= 3000`.

---

2. Hash Set Based 2Sum for Each Fixed Element

For each `i`, use a hash set to find pairs that sum to `-nums[i]`.

Complexity:

```
Time: O(n^2)
Space: O(n)
```

This can work, but duplicate handling is more awkward. You often need to store sorted triplets in a set to remove duplicates.

---

3. Sorting + Two Pointers

This is the standard and preferred approach.

Complexity:

```
Time: O(n^2)
Space: O(1) auxiliary space
```

It is efficient, clean, and handles duplicates naturally.